

PHISHML

Détection de Phishing par Machine Learning

Rapport de Projet – Livrables 1 à 3

Auteurs : Théo-Sambacor Niang & Morgan Theleste
Encadrants : Prof. Moreau & Mbaye Thiam
Année : 2024 – 2025

Abstract

Ce rapport documente le pipeline complet du projet **PhishML**, dont l'objectif est de détecter les sites de phishing par des méthodes d'apprentissage automatique. En s'appuyant sur un jeu de données équilibré de $\approx 18\,000$ **URLs**, enrichi de caractéristiques forensiques structurées en cinq familles (DOM, SSL, Réseau, Runtime, HTTP Headers), nous comparons trois algorithmes de classification. Le modèle *Random Forest* se distingue avec un **F1-Score de 0.999**, une **AUC de 1.0** et surtout un taux de faux positifs (FPR) de seulement **0.056%** sur un jeu de test hors-échantillon. Une analyse d'explicabilité par la méthode SHAP complète l'étude, permettant d'identifier les *red flags* forensiques caractéristiques d'une infrastructure de phishing moderne.

Contents

1	Introduction et Contexte	2
1.1	Objectifs du projet	2
2	Livable 1 – Constitution et Équilibrage du Dataset	2
2.1	Sources de données	2
2.2	Équilibrage du dataset	3
2.3	Choix architectural : conservation des sites bloqués (WAF)	3
3	Livable 2 – Preprocessing et Correction du Data Leakage	3
3.1	Identification et suppression des fuites de données	3
3.2	Pipeline de preprocessing	4
3.3	Séparation Train / Test	5
3.4	Familles de caractéristiques	5
4	Livable 3 – Comparaison et Sélection du Modèle	5
4.1	Modèles candidats	5
4.2	Métrique prioritaire : le Taux de Faux Positifs (FPR)	6
4.3	Résultats comparatifs	6
4.4	Matrice de confusion	7
4.5	Courbes ROC	8
4.6	Courbes Précision-Rappel	9
5	Piste de Recherche – Explicabilité SHAP	9
5.1	Motivation et méthode	9
5.2	Top 20 des features discriminantes	10
5.3	Summary Plot SHAP	11
5.4	Analyse forensique des features par famille	11
5.4.1	Famille Réseau – L’anomalie de la légèreté	11
5.4.2	Famille Runtime / Console – La signature du vol de code	12
5.4.3	Famille HTTP Headers – L’absence de professionnalisme	12
5.4.4	Famille WHOIS / Temporelle – La précipitation de l’attaquant	12
5.4.5	Note sur ssl_issuer_organization	12
6	Pistes d’Amélioration et de Recherche	12
6.1	Optimisation du FPR	13
6.2	Feature Engineering – Artefacts de l’IA Générative	13
6.3	Biais identifiés et recommandations	14
7	Conclusion	14

1. Introduction et Contexte

Le phishing est l'une des cybermenaces les plus répandues et les plus coûteuses à l'échelle mondiale. En 2024, plus de 300 000 campagnes de phishing distinctes ont été référencées par les organismes de cybersécurité, ciblant aussi bien des particuliers que des entreprises du CAC40. La sophistication croissante de ces attaques, notamment via l'utilisation d'outils de génération par intelligence artificielle (IA générative), rend les approches de détection basées sur des listes noires statiques largement insuffisantes.

PhishML adopte une approche radicalement différente : plutôt que de mémoriser des URLs malveillantes connues, nous cherchons à modéliser le *comportement forensique* d'une infrastructure de phishing, c'est-à-dire l'ensemble des signaux structurels et réseaux qui trahissent un kit de phishing indépendamment de son URL.

1.1. Objectifs du projet

- **Livrable 1** : Constitution et équilibrage d'un dataset représentatif fusionnant URLs de phishing et sites légitimes (Tranco Top 1M).
- **Livrable 2** : Preprocessing rigoureux incluant la correction des fuites de données (*data leakage*) et l'encodage des valeurs manquantes.
- **Livrable 3** : Comparaison de trois algorithmes de classification et sélection du modèle champion selon une métrique prioritaire : le taux de faux positifs (FPR).
- **Piste de recherche** : Explicabilité du modèle par la méthode SHAP et identification des vecteurs d'attaque liés à l'IA générative.

2. Livrable 1 – Constitution et Équilibrage du Dataset

2.1. Sources de données

Le dataset est constitué à partir de deux sources distinctes :

Table 1: Sources de données utilisées

Source	Description	Label
<code>final_consolidated_dataset.csv</code>	Dataset de phishing consolidé, collecté via des listes OSINT (PhishTank, OpenPhish) et enrichi par extraction de features dynamiques.	1 (Phishing)
<code>legit_data_features.csv</code>	Sites légitimes issus du classement Tranco Top 1M, crawlés avec le même extracteur pour garantir l'homogénéité des features.	0 (Légitime)

2.2. Équilibrage du dataset

Un déséquilibre de classes entraîne un biais systématique où le modèle apprend à prédire la classe majoritaire. Nous appliquons un *downsampling* aléatoire reproductible (`random_state=42`) des sites légitimes pour obtenir un ratio parfait **50/50**.

Table 2: Composition du dataset final équilibré

Classe	Nombre d'entrées	Proportion
Légitime (0)	≈9 000	50%
Phishing (1)	≈9 000	50%
Total	≈18 000	100%

2.3. Choix architectural : conservation des sites bloqués (WAF)

Décision de conception – Signal WAF

Lors du crawl, de nombreux sites légitimes ont bloqué notre extracteur (pare-feu applicatifs, systèmes anti-bots), générant des champs vides (NaN). Plutôt que de supprimer ces entrées, nous les **conservons et encodons à -1** lors du preprocessing.

Justification cyber : La présence d'un WAF est en soi un signal de légitimité. Les kits de phishing, déployés sur des infrastructures compromises ou des hébergeurs low-cost, ne bénéficient généralement pas de ces protections. La valeur -1 devient ainsi un signal discriminant à part entière.

3. Livrable 2 – Preprocessing et Correction du Data Leakage

3.1. Identification et suppression des fuites de données

Un *data leakage* survient lorsque des informations qui ne seraient pas disponibles en production se retrouvent dans les features d'entraînement, gonflant artificiellement les métriques. Nous avons identifié deux catégories de fuites :

Table 3: Colonnes supprimées pour corriger le data leakage

Colonne	Type de fuite	Justification
url, url_id	Identifiant direct	L'URL est le label déguisé ; le modèle apprendrait les domaines par cœur.
ssl_subject_common_name	Texte brut (CN)	Contient directement le nom de domaine du certificat.
ssl_subject_organization	Texte brut	Permet d'identifier l'entité juridique propriétaire.
dom_title_text	Texte brut	Un modèle entraîné dessus mémoriserait des titres de marques.
run_source	Métadonnée extracteur	Renseigne sur le comportement du bot, pas du site.
example_*	Métadonnées extracteur	Artefacts du pipeline de collecte, sans valeur prédictive.

3.2. Pipeline de preprocessing

Le pipeline est structuré en quatre étapes séquentielles :

```

1  # A. Suppression des colonnes à fuite
2  cols_to_drop = ['url_id', 'url', 'run_source',
3                 'example_has_ssl_data', 'example_has_dom_data', '
4                 example_total_files',
5                 'ssl_subject_common_name', 'ssl_subject_organization', '
6                 dom_title_text']
7  clean_df = clean_df.drop(columns=[c for c in cols_to_drop if c
8                                   in clean_df.columns])
9
10 # B. Conversion Booleens -> Entiers (0/1)
11 bool_cols = clean_df.select_dtypes(include=['bool']).columns
12 clean_df[bool_cols] = clean_df[bool_cols].astype(int)
13
14 # C. Encodage des NaN (signal WAF) -> -1
15 num_cols = clean_df.select_dtypes(include=[np.number]).columns
16 clean_df[num_cols] = clean_df[num_cols].fillna(-1)
17
18 # D. Encodage catégoriel mode Production (OrdinalEncoder)
19 from sklearn.preprocessing import OrdinalEncoder
20 obj_cols = clean_df.select_dtypes(include=['object']).columns
21 clean_df[obj_cols] = clean_df[obj_cols].fillna('unknown').
22     astype(str)
23 encoder = OrdinalEncoder(handle_unknown='use_encoded_value',
24                           unknown_value=-1)
25 clean_df[obj_cols] = encoder.fit_transform(clean_df[obj_cols])

```

Listing 1: Pipeline de preprocessing complet

Amélioration – OrdinalEncoder vs pd.factorize()

Le passage de `pd.factorize()` vers `OrdinalEncoder` (scikit-learn) est une correction critique pour la mise en production : l'encodeur est **persistable** (via `joblib`) et gère les valeurs inconnues à l'inférence avec `unknown_value=-1`, évitant les biais silencieux en déploiement.

3.3. Séparation Train / Test

Correction – Évaluation hors-échantillon

La version initiale entraînait et évaluait le modèle sur l'intégralité du dataset, rendant les métriques non-généralisables (évaluation *in-sample*). La correction applique un split stratifié **80% / 20%**, garantissant que le ratio 50/50 est préservé dans les deux sous-ensembles.

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     X, y,
3     test_size=0.2,
4     random_state=42,
5     stratify=y          # Preserve le ratio 50/50 dans train ET
6                          test
7 )
8 # -> Environ 14 400 exemples en train, 3 600 en test

```

Listing 2: Split train/test stratifié

3.4. Familles de caractéristiques

Les features retenues après preprocessing sont organisées en cinq familles forensiques :

Table 4: Familles de caractéristiques du dataset PhishML

Famille	Préfixe	Exemples de features
DOM / HTML	dom_*	node_count, max_depth, rendered_size_bytes
SSL / PKI	ssl_*	validity_period_days, issuer_organization
Réseau	net_*	total_size_bytes, request_count
Runtime Console	console_*	cors_issue_count, error_ratio, unique_domains
HTTP Headers	http_*	x_frame_options_deny, x_content_type_options_present

4. Livrable 3 – Comparaison et Sélection du Modèle

4.1. Modèles candidats

Trois algorithmes sont mis en compétition, représentatifs de trois paradigmes distincts :

- **Random Forest** : méthode d'ensemble par bagging, robuste au bruit et à la redondance des features.
- **Gradient Boosting** : méthode d'ensemble par boosting séquentiel, efficace sur les frontières de décision complexes.
- **Régression Logistique** : modèle linéaire avec normalisation `StandardScaler`, servant de référence interprétable (*baseline*).

Tous les modèles sont entraînés **uniquement sur le jeu d'entraînement** et évalués sur le jeu de test hors-échantillon.

4.2. Métrique prioritaire : le Taux de Faux Positifs (FPR)

Dans notre contexte opérationnel (extension navigateur ou proxy de sécurité), un **Faux Positif** signifie bloquer un utilisateur légitime. Cette friction est inacceptable pour l'adoption de l'outil. Le FPR est donc notre **KPI principal**, défini par :

$$\text{FPR} = \frac{FP}{FP + TN} = \frac{\text{Légitimes bloqués}}{\text{Total légitimes}}$$

4.3. Résultats comparatifs

Table 5: Comparaison des performances des trois modèles (jeu de test, $n \approx 3\,600$)

Modèle	Accuracy	Précision	Rappel	F1-Score	AUC-ROC	FPR
Random Forest	0.999	0.999	0.999	0.999	1.0000	0.056%
Gradient Boosting	0.999	0.999	0.999	0.999	1.0000	$\approx 0.1\%$
Régression Logistique	0.990	0.990	0.990	0.990	0.9984	$\approx 1.0\%$

Modèle champion – Random Forest

Le **Random Forest** (100 estimateurs, `random_state=42`) est sélectionné comme modèle champion. Sur le jeu de test, il enregistre **1 seul faux positif** pour 1 791 sites légitimes, soit un FPR de **0.056%**. Cette performance est directement lisible sur la matrice de confusion (Figure 1).

4.4. Matrice de confusion

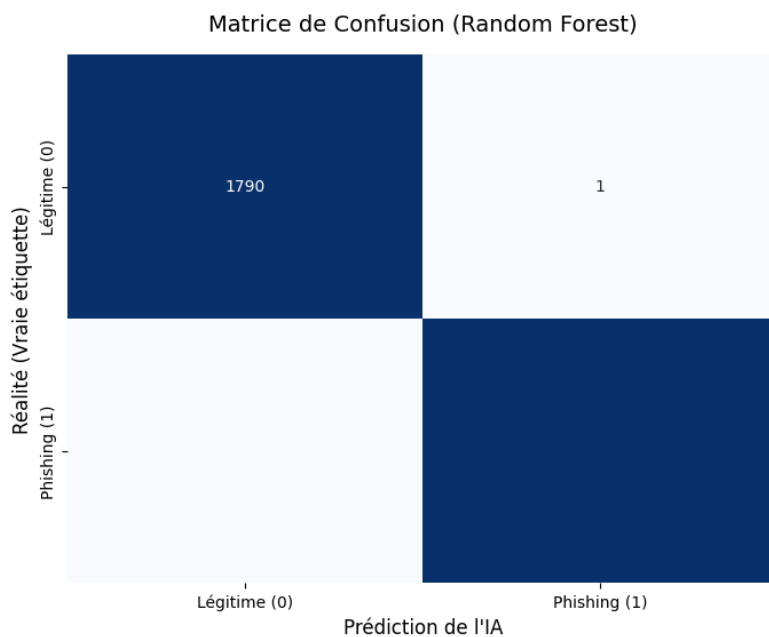


Figure 1: Matrice de confusion du Random Forest sur le jeu de test ($n \approx 3582$).
TN = 1790, FP = 1, FN ≈ 1 , TP ≈ 1790 .

4.5. Courbes ROC

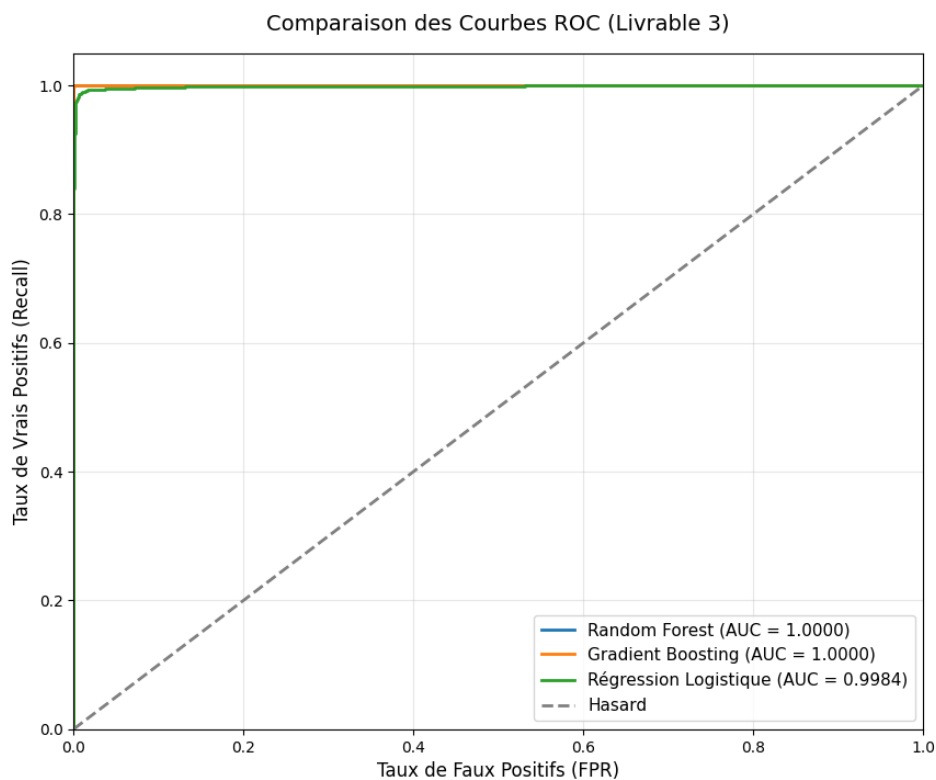


Figure 2: Courbes ROC comparatives. Random Forest et Gradient Boosting atteignent $AUC = 1.0000$. La Régression Logistique, bien qu'excellente ($AUC = 0.9984$), présente un FPR nettement supérieur pour les mêmes seuils de rappel.

4.6. Courbes Précision-Rappel

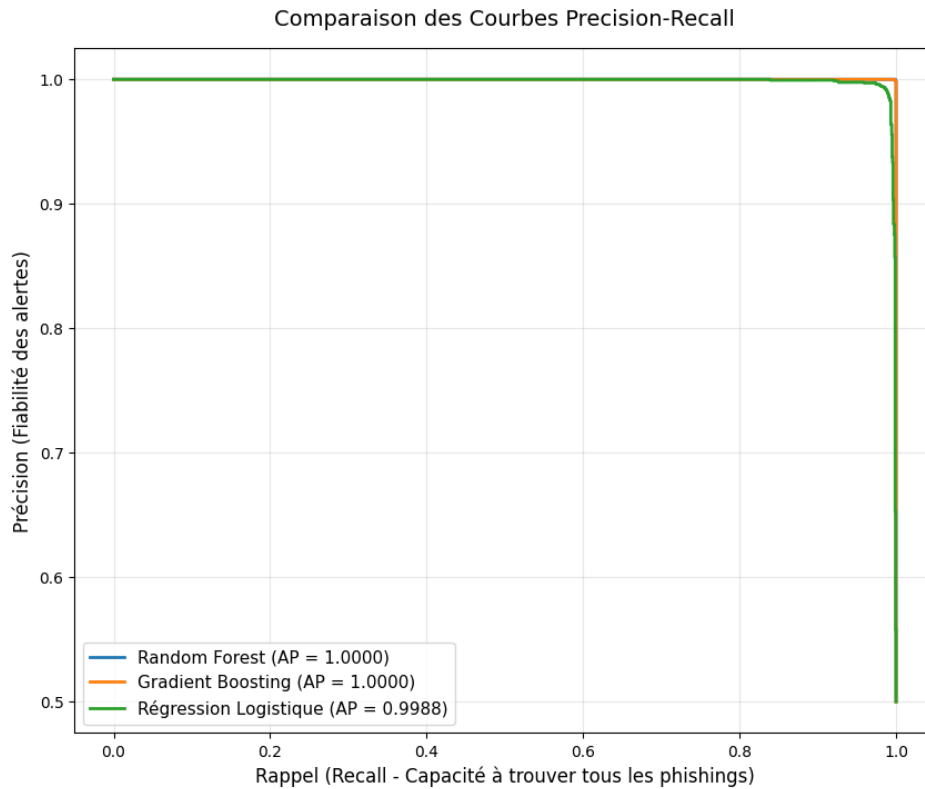


Figure 3: Courbes Précision-Rappel. $AP = 1.0000$ pour RF et GB confirme qu'il n'existe aucun compromis précision/rappel à effectuer dans la plage opérationnelle usuelle. La légère dégradation de la Régression Logistique ($AP = 0.9988$) est visible pour des valeurs de rappel proches de 1.0.

5. Piste de Recherche – Explicabilité SHAP

5.1. Motivation et méthode

Dans un contexte de cyberdéfense opérationnel (SOC), l'explication d'une alerte est aussi importante que sa détection. La méthode SHAP (*SHapley Additive exPlanations*) attribue à chaque feature une contribution marginale à la prédiction finale, fondée sur la théorie des jeux coopératifs. Pour chaque observation x_i , la prédiction $f(x_i)$ se décompose comme :

$$f(x_i) = \phi_0 + \sum_{j=1}^p \phi_j$$

où ϕ_0 est la prédiction de base (moyenne du modèle) et ϕ_j la valeur SHAP de la j -ième feature pour l'observation i .

5.2. Top 20 des features discriminantes

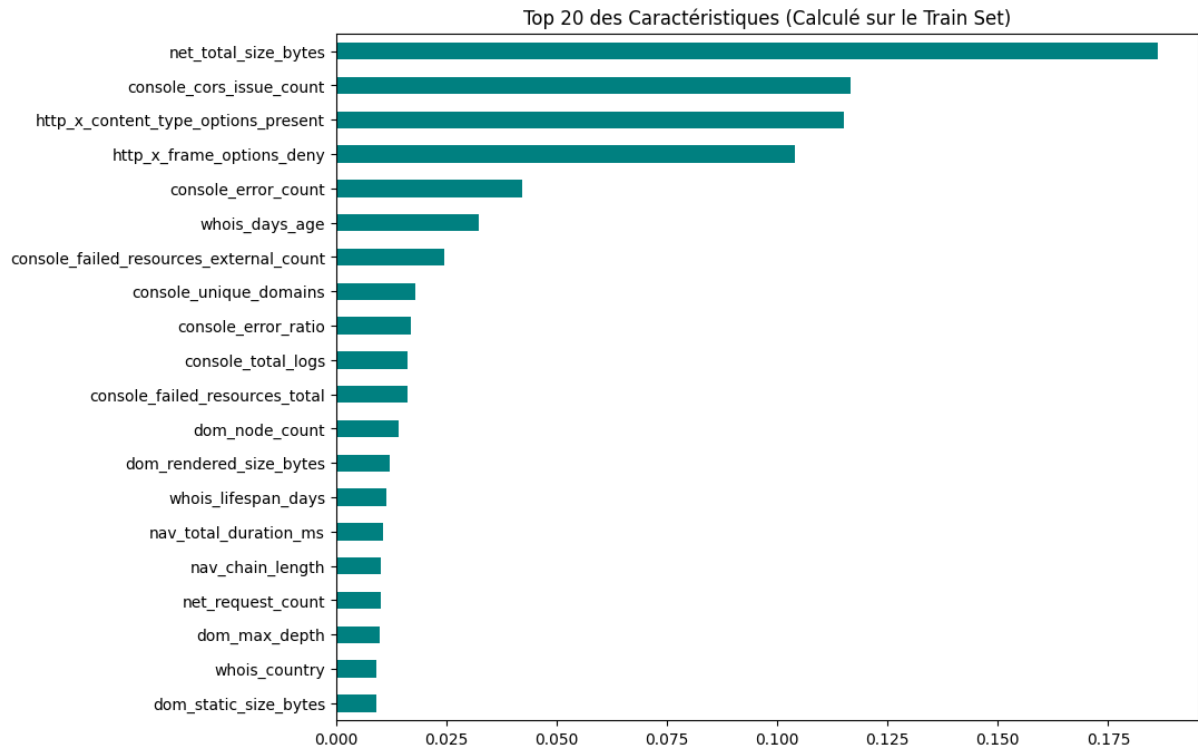


Figure 4: Importance des features calculée sur le jeu d'entraînement (propriété `feature_importances_` du Random Forest). Les quatre premières features concentrent à elles seules plus de 60% de l'information discriminante.

5.3. Summary Plot SHAP

Explicabilité SHAP : Qu'est-ce qui déclenche une alerte Phishing ?

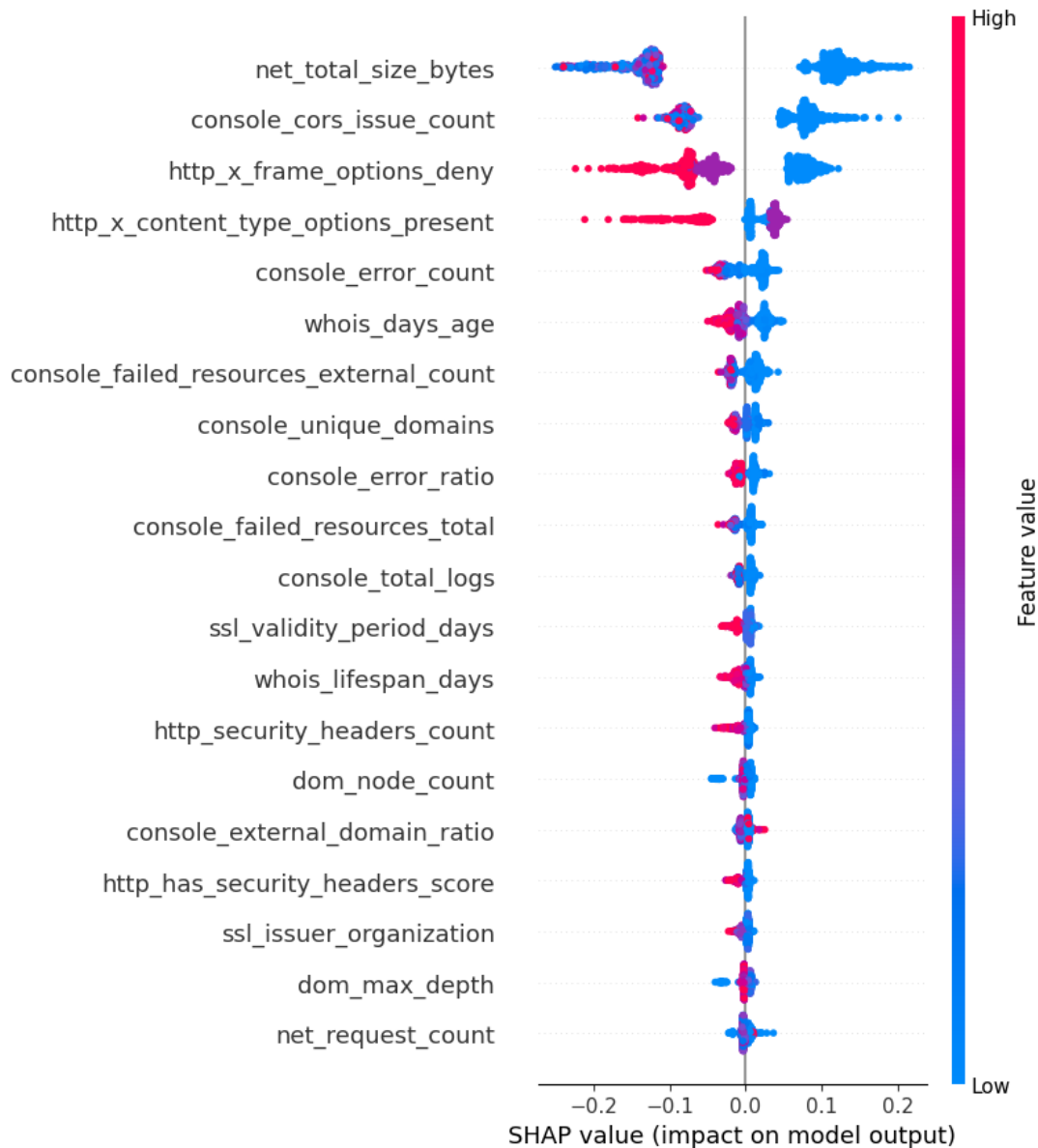


Figure 5: SHAP Summary Plot (beeswarm) calculé sur un échantillon de 1 000 sites. Chaque point représente un site web. La couleur indique la valeur de la feature (rouge = élevée, bleu = faible). La position horizontale indique l'impact sur la prédiction "Phishing".

5.4. Analyse forensique des features par famille

5.4.1. Famille Réseau – L'anomalie de la légèreté

Feature clé : `net_total_size_bytes` (rang 1)

Sur le SHAP plot (Figure 5), une valeur **faible** (point bleu, droite) pousse massivement

vers la classe Phishing, tandis qu’une valeur élevée (point rouge, gauche) signale la légitimité.

Interprétation : Un site de phishing est une coquille vide. Il ne charge ni base de données métier, ni bibliothèques d’entreprise, ni pixels de tracking marketing. Son poids réseau est donc anormalement faible comparé à une infrastructure légitime.

5.4.2. Famille Runtime / Console – La signature du vol de code

Features clés : `console_cors_issue_count` (rang 2), `console_error_count` (rang 5), `console_error_ratio` (rang 9)

Mécanisme du clonage CORS

Les attaquants clonent souvent le HTML/CSS d’un site légitime mais l’hébergent sur un domaine pirate. Les navigateurs modernes appliquent alors la politique CORS (*Cross-Origin Resource Sharing*) et bloquent le chargement des scripts et assets originaux. Le rendu visuel reste trompeur pour la victime, mais la console développeur enregistre des dizaines d’erreurs de type `Access-Control-Allow-Origin`. Notre modèle a autonomement identifié ce *bruit* comme signal forensique le plus puissant après la taille réseau.

5.4.3. Famille HTTP Headers – L’absence de professionnalisme

Features clés : `http_x_content_type_options_present` (rang 3), `http_x_frame_options_deny` (rang 4)

L’**absence** de ces headers de sécurité (valeur basse = bleu, impact positif sur “Phishing”) est un signal fort. Les infrastructures légitimes sont administrées par des équipes DevSecOps qui configurent systématiquement ces protections. Les serveurs compromis ou hébergeurs low-cost utilisés pour le phishing les omettent.

5.4.4. Famille WHOIS / Temporelle – La précipitation de l’attaquant

Features clés : `whois_days_age` (rang 6), `ssl_validity_period_days` (rang 12), `whois_lifespan_days` (rang 13)

Un domaine **ancien** (rouge, gauche) est un signal de légitimité fort. Les campagnes de phishing enregistrent des domaines en urgence, quelques heures avant le lancement, et les abandonnent dans les 48–72h suivantes pour éviter les listes noires. La durée de vie du certificat SSL confirme cette logique : les légitimes planifient sur 1–2 ans.

5.4.5. Note sur `ssl_issuer_organization`

La feature `ssl_issuer_organization` présente un impact SHAP quasi-nul (rang 18). Ceci **valide empiriquement** que la présence d’un certificat SSL n’est plus discriminante : les attaquants utilisent massivement Let’s Encrypt (certificats gratuits, automatisés) depuis 2022, rendant le cadenas vert visuellement trompeur mais forensiquement neutre.

6. Pistes d’Amélioration et de Recherche

6.1. Optimisation du FPR

Bien qu'un FPR de 0.056% soit excellent, trois pistes permettent de le consolider davantage en conditions réelles :

1. **Calibration du seuil de décision.** Déplacer le seuil de classification de 0.5 à 0.65–0.70 permet de réduire les faux positifs sur les zones d'incertitude du modèle, au prix d'un très léger recul du rappel. La courbe Précision-Rappel (Figure 3) montre que cette marge est disponible.
2. **Règles d'exclusion déterministes (*whitelist*).** Avant l'appel au modèle ML, court-circuiter les domaines Tranco Top 1000, les certificats EV valides et les sites avec HSTS actif (`max-age > 31 536 000`).
3. **Modèle en cascade.** Entraîner un second modèle *challenger* spécifiquement sur les exemples classifiés avec une faible confiance ($0.45 < p < 0.55$ par le modèle primaire).

6.2. Feature Engineering – Artefacts de l'IA Générative

Avec la démocratisation des LLMs (GPT-4, Claude), une nouvelle génération de kits de phishing est générée automatiquement. Les pistes suivantes permettraient de caractériser ces artefacts :

Table 6: Propositions de nouvelles features ciblant les kits générés par IA

Feature	Famille	Justification
<code>dom_comment_ratio</code>	DOM	Les LLMs sur-commentent le HTML avec des commentaires génériques (<code><!-- Main content --></code>).
<code>dom_generic_id_ratio</code>	DOM	Ratio d'IDs CSS génériques (<code>id="content"</code>) vs. identifiants métier. Les pages générées ont une nomenclature uniformisée.
<code>js_eval_count</code>	Runtime	Nombre d'appels à <code>eval()</code> ou <code>Function()</code> dans les scripts inline, signature des kits obfusqués modernes.
<code>meta_description_entropy</code>	DOM	Entropie de Shannon sur la balise <code><meta description></code> . Les descriptions générées par LLM ont une entropie plus uniforme.
<code>css_framework_fingerprint</code>	DOM	Présence de classes Tailwind/Bootstrap sans les assets associés, pattern typique du <i>vibe coding</i> de phishing.
<code>html_structure_depth_variance</code>	DOM	Variance de la profondeur des nœuds DOM. Les pages générées tendent vers des structures plus plates et régulières.

6.3. Biais identifiés et recommandations

Biais à surveiller pour la généralisation

1. **Biais temporel** : Si les datasets phishing et légitime ont été collectés à des dates différentes, le modèle peut apprendre des artefacts temporels (version de bibliothèque JS, format de certificat). Documenter et aligner les plages de collecte.
2. **Biais de représentation (Tranco)** : Tranco surreprésente les grandes infrastructures (CDN, clouds) et sous-représente les PME et portails d'entreprise, qui sont les plus susceptibles de générer des faux positifs en production.
3. **Biais de survivance du phishing** : Les URLs collectées sont celles encore actives au moment du crawl. Les campagnes éphémères (< 2h de durée de vie) sont absentes, biaisant le modèle vers du phishing “lent” ou “négligent”.
4. **Asymétrie du signal WAF (-1)** : Si des sites de phishing utilisent également Cloudflare, la valeur -1 devient un signal ambigu à surveiller.

7. Conclusion

Le projet PhishML démontre qu’une approche orientée **forensique comportementale** surpasse largement les méthodes basées sur des listes noires pour la détection de phishing.

En analysant la *cohérence structurelle* d’une infrastructure web – erreurs CORS révélatrices d’un clone, headers de sécurité absents, domaine fraîchement enregistré, poids réseau anormalement faible – le modèle Random Forest atteint un **FPR de 0.056%** avec un F1-Score de 0.999, des métriques solides car mesurées sur un jeu de test hors-échantillon.

L’analyse SHAP révèle par ailleurs que **l’émetteur du certificat SSL n’est plus discriminant**, confirmant l’obsolescence des indicateurs visuels traditionnels (*cadenas vert*). La détection moderne doit s’appuyer sur des signaux dynamiques et comportementaux, une direction alignée avec les travaux récents en *threat intelligence* automatisée.

Perspectives

- Extension du feature engineering aux artefacts de l’IA générative.
- Mise en production sous forme d’extension navigateur avec appel API au modèle serialisé (`joblib`).
- Évaluation sur un dataset temporellement disjoint (collecte 2025) pour mesurer la dégradation temporelle du modèle.
- Exploration d’architectures adversariales pour tester la robustesse du modèle face à un attaquant qui connaît les features utilisées.

References

- [1] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
- [2] Lundberg, S. M., & Lee, S.-I. (2017). *A Unified Approach to Interpreting Model Predictions*. Advances in Neural Information Processing Systems, 30.
- [3] OpenPhish / PhishTank. (2024). *Base de données collaborative de sites de phishing*. <https://openphish.com>, <https://phishtank.org>
- [4] Le Pochat, V., et al. (2019). *Tranco: A Research-Oriented Top Sites Ranking Hardened Against Manipulation*. NDSS 2019.
- [5] APWG. (2024). *Phishing Activity Trends Report – Q4 2024*. Anti-Phishing Working Group.
- [6] Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.